

One weak prompt, five documented revisions

Start from "Explain neural networks." and take it to expert level in five revisions, each adding one engineering ingredient. Score every response against the same rubric, so improvement is measured rather than felt.

Scaffolded lab

25 minutes

One frontier model

Measured, not felt

Before you start

The rubric · given

1 · five revisions

2 · the log and the chart ...

3 · context-window discipline

Reflection

If you finish early

Before you start

Open the provided `lab18.ipynb` and one frontier model of your choice in a browser tab. The logging and the chart are given; the prompts and the scores are yours.



By the end of this lab you can

- Name the parts of a prompt: task, audience, context, constraints, output format, verification.
- Improve a prompt one ingredient at a time and record the effect.
- Score responses against a fixed rubric, and remove what does not help.

LAB 18

The rubric · given

Score each response 0–5 on each criterion. Decide what a 5 looks like **before** you read any response.

Criterion	0 looks like	5 looks like
Level	for nobody in particular	exactly right for a BS student who has finished session 14
Accuracy	at least one false claim	nothing you could catch with the course notes
Structure	a wall of text	the structure you asked for
Examples	none, or generic	concrete, and at least one uses the 2–3–1 network
Verification	confident throughout	says what to check, or where it is unsure

TASK 1 · 15 MINUTES · THE MAIN TASK

five revisions

1

Each revision adds **one** ingredient to the one before. Paste the full prompt text, send it to the same model in a new chat each time, and fill in the scores.

your turn

PYTHON

```
prompts = {
    0: "Explain neural networks.",
    1: "", # TODO + audience and goal
    2: "", # TODO + context: what the reader already knows
    3: "", # TODO + constraints and scope
    4: "", # TODO + output structure and example requirements
    5: "", # TODO + verification: what to flag, what to check
}

ingredient = {0: "weak baseline", 1: "audience + goal", 2: "context", 3: "constraints + scope",
              4: "structure + examples", 5: "verification"}

scores = { # TODO: (level, accuracy, structure, examples, verification), each 0-5
    0: (0, 0, 0, 0, 0), 1: (0, 0, 0, 0, 0), 2: (0, 0, 0, 0, 0),
    3: (0, 0, 0, 0, 0), 4: (0, 0, 0, 0, 0), 5: (0, 0, 0, 0, 0),
}
```

the log and the chart · given

2

given

PYTHON

```
import matplotlib.pyplot as plt

for k in prompts:
    added = prompts[k][len(prompts[k - 1]):].strip() if k and prompts[k].startswith(prompts[k - 1]) else prompts[k]
    print(f"rev {k} · {ingredient[k]:<22} total {sum(scores[k]):>2}/25 words {len(prompts[k].split()):>3}")

assert all(prompts[k].strip() for k in prompts), "every revision needs its prompt text"
assert all(prompts[k] != prompts[k - 1] for k in range(1, 6)), "each revision must change something"

plt.plot(list(prompts), [sum(scores[k]) for k in prompts], "o-")
plt.xticks(list(prompts), [ingredient[k] for k in prompts], rotation=30, ha="right")
plt.ylabel("rubric total / 25"); plt.xlabel("revision"); plt.title("What each ingredient bought")
plt.tight_layout(); plt.show()
```

what it should print

OUTPUT

rev 0 · weak baseline	total 7/25	words 3
rev 1 · audience + goal	total 11/25	words 18
rev 2 · context	total 12/25	words 29
rev 3 · constraints + scope	total 15/25	words 45
rev 4 · structure + examples	total 20/25	words 55
rev 5 · verification	total 24/25	words 74

context-window discipline

3

Copy revision 5 into the cell below and label each sentence as **task**, **context**, **constraint**, **output format**, **example**, or **verification**. Then **delete one sentence** that did not earn its place, resend, and rescore. Better prompting is not longer prompting.

your turn

PYTHON

```
final_prompt = ""      # TODO: revision 5 with one sentence removed
final_score = (0, 0, 0, 0, 0)    # TODO
print("words:", len(final_prompt.split()), " total:", sum(final_score), "/25")
```

Reflection



Write it in the notebook, in your own words

Contrast **fine-tuning** (session 26: changes the parameters) with **prompting** (today: changes the instructions at the moment of use). Give one situation in which each is clearly the better choice.

Your answer:

If you finish early

A. Send revision 5 to a **second** frontier model and score it with the same rubric. Is your best prompt portable, or was it tuned to one model?

your turn

PYTHON

```
# TODO (optional extension A)
```